

Shadow — Threat Model (v3, matches Shadow v7.0.0)

Shadow — Threat Model

Status: living document · **v3 re-cut (P3-12)** · matches Shadow **v7.0.0** (2026-08-16) **Companion:** run `shadow doctor --privacy` to verify the egress posture of *your* config at any time.

Shadow is a coding agent: it runs a language model in a loop with tools that read files, write files, and execute shell commands. This document states — honestly — what Shadow protects, **how**, and what it does **not** protect. Claims here map to real mechanisms in the source; limits are stated with the same prominence as guarantees. If a sentence in our marketing ever disagrees with this document, this document wins.

1. The one-paragraph philosophy

The model is untrusted input. Everything a model emits — text, tool calls, "reasoning" — is treated the way a browser treats a web page: potentially adversarial output that must be *contained*, not trusted. Containment comes from six mechanisms working together: the **workspace jail** (file tools), the **OS sandbox** (shell *and* MCP stdio children), the **approval gate** (autonomy levels), **scoped secrets** (env allowlist + encrypted vault), the **egress broker** (one audited chokepoint for every outbound request, with an optional quarantine tier over model-authored fetches), and the **untrusted-content envelopes** (web/MCP/plugin bytes enter the context visibly fenced). Nothing in Shadow's safety story depends on the model being well-behaved.

2. Trust model

Thing	Trust level	Why
The model's output	Untrusted	May be wrong, manipulated, or adversarial (prompt injection)
Fetches web content / search results / MCP replies	Untrusted data	Enveloped in containment markers (§3.11); never instructions; limits in §4.2
A cloned repo's <code>shadow.config.json</code>	Untrusted	De-fanged on load (§3.7)
A cloned repo's <code>.shadow/agents/*.md</code> + skills	Untrusted, loaded raw	Loaded from the workspace with NO install or enable step (unlike plugins, §4.13); an agent def's body becomes that sub-agent's system prompt and its <code>tools</code> list arms it (§3.13, §4.14)
Installed plugins (<code>~/.shadow/plugins/<name></code>)	Data-only, user-vetted	Markdown-only copy, executable manifest keys refused, installed DISABLED; enable is an explicit act (§4.13)
A configured plugin index	Untrusted listings	Display-only entries; <code>add <name></code> URLs re-pass the git scheme allowlist; optional fail-closed signature (§4.13)
Your global <code>~/.shadow/</code> config	Trusted by design	It is <i>your</i> file on <i>your</i> machine (§4.9)
Configured MCP servers	Trusted once added	Stdio children jailed on macOS/Linux (network off unless granted); your user on Windows; vet before adding (§3.8, §4.7)
An ACP editor that spawns <code>shadow acp</code> (Zed et al.)	Trusted local process	It spawns Shadow and exchanges JSON-RPC over stdio; tool approval is bridged to it, but the jail, run lock, and fail-closed floor still apply (§3.12). A trojaned editor is out of scope, like the local-model runtime (§4.8)

Thing	Trust level	Why
The provider endpoint you configure	Fully trusted with your data	The conversation goes there — that is the product working as designed (§4.8)
The local model runtime (llama.cpp, Ollama)	Trusted infrastructure	A trojaned runtime is out of scope (§4.8)
Env vars, CLI flags, the installer	Trusted	Standard local-machine trust (§4.11)

Assets protected: your API keys; files outside the workspace; the workspace itself (from catastrophic commands); your network posture (SSRF); the privacy of the session (no telemetry).

3. Mechanisms and their real guarantees

3.1 Workspace jail — file tools

Every file-tool path resolves to an absolute path that must stay inside the workspace root (plus any explicitly granted `--add-dir` roots). Symlinks are collapsed against the deepest existing ancestor, so a symlink pointing outside — or a **not-yet-created** path — cannot escape. **Limit:** the jail governs *file tools only*. Shell commands are contained by the OS sandbox (§3.2), not the jail.

3.2 OS sandbox — `run_shell`

- macOS** (seatbelt): writes denied everywhere except the workspace, granted dirs, and temp dirs; reads *and* writes of `~/.shadow` denied (your keys are invisible to shell commands).
- Linux** (bubblewrap): whole filesystem read-only; workspace + `/tmp` writable; `~/.shadow` masked by an empty tmpfs; PID namespace unshared (blocks `/proc/<pid>/environ` key theft).
- Windows: no OS sandbox exists.** `run_shell` is unconfined. This is stated, not hidden.

Failure policy (P2-12): when the sandbox was *requested* but the host has no tool to enforce it (`sandbox-exec` / `bwrap` missing), Shadow never silently fakes confinement — it prints a startup banner, reports the state in `shadow doctor` , `doctor --privacy` , and `/status` , and applies `sandboxFailurePolicy` (global config only — a project file cannot set it):

- `auto` (*default*) — an unconfined `run_shell` stops at the approval gate, like the autonomy floor: a session approval, allow-rule, or the read-only fast path may suppress later prompts.
- `fail-closed` — the gate **never bends**: every unconfined call asks every time (denylist tier).
- `warn` — pre-P2-12 behavior: no gate; a warning is folded into the tool result. An explicit waiver (`sandbox: "off"` , `--no-sandbox` , `--yolo` , `--autonomy full` at launch) never escalates — the operator already chose unconfinement. **Remaining limits:** reads outside the workspace are **allowed by design** on macOS/Linux (builds need headers, `node_modules`, etc.) — *except* the curated credential denylist (35 entries, `SECRET_READ_DENY` in `src/safety/sandbox.ts`), which seatbelt enforces as explicit `deny file-read*` rules and bubblewrap as masked tmpfs mounts. It is tiered: SSH/cloud-CLI/package-manager credentials (`~/.ssh` , `~/.aws` , `~/.gnupg` , `~/.config/gh` , `~/.netrc` , `~/.kube` , `~/.npmrc` , `~/.pypirc` , `~/.git-credentials` , `~/.config/gcloud` , `~/.azure` , `~/.config/huggingface` , `~/.m2/settings.xml` , `~/.docker/config.json`); the password-manager + browser + OS-keychain tier (`~/.password-store` , Chrome/Chromium/Edge/Firefox/Brave profile dirs, `~/.Library/Keychains` , `~/.local/share/keyrings` , 1Password app data — whole-directory denies, deliberately: the secret files move between versions, and there is no per-command waiver for a single read); and the third-party-store tier (`~/.config/rclone` , `~/.vault-token` , `~/.oci` , Electrum/Bitcoin wallet dirs). Anything outside that enumeration remains readable (`~/.gitconfig` , shell rc files, package-manager caches — useful, not secret); the structural inversion (`deny $HOME` reads by default, allow workspace + grants) is deferred — it breaks legitimate tooling reads until the grant model catches up. Pair with `sandboxNetwork: false` or `--offline` to cut the exfiltration path. Network egress from the sandbox is allowed by default (installs need it).

3.3 Secrets — scoped env + encrypted vault

Shell commands receive an **allowlist** of ~8 environment variables (`PATH` , `HOME` , `TERM` , ...). Provider API keys are **never** in a subprocess environment. At rest, keys live either in `credentials.json` (0600) or the **encrypted vault**: `script` ($N=2^{16}$, $r=8$, $p=1$) → AES-256-GCM, authenticated — a wrong password or a tampered file fails closed. The OS keychain caches the *derived key*, never the password. **Limits**: vault security is bounded by master-password strength (offline brute force of a stolen `vault.enc` is feasible against weak passwords). Anyone inside your unlocked OS session can use the keychain-cached key (that is what "unlocked session" means). `SHADOW_VAULT_PASSWORD` for headless use sits in the environment in plaintext — prefer the keychain on interactive machines.

3.4 Network guard — web tools

`web_fetch` / `web_search` refuse non-HTTP schemes and any host resolving to loopback, RFC-1918, link-local, CGNAT, v6-transition addresses, or cloud-metadata **in every spelling** — `169.254.169.254` plus its IPv4-mapped/compat/NAT64/6to4/Teredo v6 encodings, and AWS IMDSv6 `fd00:ec2::254` (which sits inside `fc00::7` ULA and would otherwise classify as local). Every resolved address is validated **before connect**; on **Node runtimes** the connection is additionally **pinned to the validated IP SET** via a dedicated undici dispatcher (defeats DNS rebinding; multi-A failover survives). Netguard-tier traffic forces `redirect: 'manual'` and re-validates every hop itself (5-hop cap); operator-tier auto-follow redirects that cross hosts fall back to real DNS for the new host instead of dialing the original pin set. Since v6.7.0 these requests — and **every** outbound request Shadow makes — flow through one egress broker (`src/safety/egress.ts`): raw `fetch` / undici imports outside the broker fail lint, and each decision is journaled to a local receipt (`~/shadow/egress.log` , 0600; `shadow egress` prints it). **Runtime scope of the pin**: the shipped binary is built with Bun, whose `undici` import resolves to a stub — the socket-layer pin therefore exists only when Shadow runs on Node (`npm /dev`). In the binary, validation still happens before connect (same tiers, same refusals), but no pin is held on the socket afterward, so a DNS-rebinding adversary has a window between validation and connect. This is a pre-existing condition of the binary build, not a v6.7.0 regression. **Quarantine tier (P3-08, v6.24.0)**: on top of the broker sits an allowlist over *model-authored* destinations — exactly the `web / search / image` purposes (URLs the model produced). Operator-tier traffic (provider, MCP, oauth, updates, local probes) is never quarantined. The policy (`egress` in global config; a project file cannot set it) is `observe` (*default*) — an off-allowlist fetch proceeds but is **flagged** on the receipt — or `enforce` — it is **denied** with a readable error. Entries match exactly or as `*.x.com` subdomains (never the apex; a bare wildcard admits nothing); `web_search` 's `duckduckgo.com` dependencies are pre-allowed. The flag/deny lands in the same receipt `shadow egress` prints and `doctor --privacy` summarizes. **Limit**: the quarantine governs where the *web tools* may fetch — it cannot see sockets opened by sandboxed shell commands or MCP children (§3.8's honest residual). A manipulated model with `network` approval can still POST data to a public endpoint from the shell; the gate, not the guard, is the control for that (§3.6).

3.5 Denylist — fat-finger guard, not a boundary

Catastrophic command patterns (`rm -rf /` , `mkfs` , `dd of=/dev/...` , fork bombs) are refused even at full autonomy. The source code itself says what we repeat here: it is **trivially bypassed by indirection** (env vars, subshells, base64). It exists to stop accidents, not attackers. The real boundary is the sandbox.

3.6 Approval gate — autonomy levels

`manual` → every tool call approved; `auto-read` → reads free, writes/exec approved; `auto-edit` (default) → edits free, exec approved; `full` → everything auto-approved except denylist hits. Every gated call passes one seam (the ApprovalGate), which is also where session and prefix approvals live. The read-only fast path (`grep` / `rg` / `find` and git read subcommands outside the roots auto-running) is honesty-hardened: an operand the analyzer cannot vouch for — a glob, tilde, variable, quoted/escaped token, or newline that could chain a second command — **demotes the call to the gate** instead of auto-running, and a granted command cannot smuggle reads or write-flags past its segment (P2-07). Residual: a URI-shaped operand (`cat https://...`) contains none of the demotion characters and, not being absolute, resolves as a relative in-workspace path — so it auto-runs; harmless today only because no read-only prefix fetches URLs. **Limit**: at `full` , containment shrinks to sandbox + jail + denylist. That is a deliberate, user-chosen trade.

3.7 Untrusted project config — drive-by defense

A cloned repo's `shadow.config.json` is stripped of every security-bearing key at load — the full set is pinned in code as `PROJECT_UNTRUSTED_KEYS` in `src/config.ts` (the live list of record — 23 keys today), and the strip is open-ended by design: anything new that gates trust joins the list. In practice a project file **cannot**: redirect your keys (`baseUrl`, `selfHosted`, `vision` upload target stripped), re-add secrets or widen the shell env (`shellEnvAllowlist`), raise autonomy, add permission rules or denylist entries, weaken the sandbox or its failure policy (`sandbox`, `sandboxNetwork`, `sandboxFailurePolicy`), swap the system prompt, widen the jail (`additionalDirectories`), widen the web-console project allowlist (`projects`), force `offline`, run startup hooks, run `statusLine` or **diagnostics** commands (command-bearing, global-only), point plugin discovery at its own index (`pluginIndexUrl` / `pluginIndexKey`), plant a named **profile** that would ride your flags (`profiles`), sweep your session history away (`sessionRetentionDays` / `Keep` — retention is archive-over-delete), or set the **egress quarantine** policy. It also cannot register MCP servers (dropped entirely), and model presets from a project file are stripped of `gguf` / `ggufServer` / `ggufArgs` — a preset that spawns a binary pre-LLM would be zero-interaction RCE. The merge itself skips `__proto__` / `constructor` / `prototype` keys (no prototype-pollution bypass of the strip, P2-11 review).

3.8 MCP — always execution-risk

MCP tools are always gated as `exec` risk. A server's self-declared `readOnlyHint` is deliberately **not trusted** — a malicious server could label `delete_files` read-only. See §4.7 for what MCP trust still means.

Stdio confinement (P3-08): stdio children spawn inside the same OS jail as `run_shell` — `workspace+/tmp` writes, credential stores denied, **network OFF by default** (a stdio server speaks over its pipes; grant per-server with `"network": true`, global config only). On macOS the seatbelt profile also denies `AF_UNIX` connects to agent sockets under `tmp` (`launchd/ssh/gpg` — (`deny network*`) covers `INET` only); on Linux the child gets a **private tmpfs** `/tmp` for the same reason. `sandbox: false` opts one server out (explicit operator choice); `sandboxFailurePolicy: fail-closed` REFUSES a child that can't be jailed instead of spawning it unconfined. Honest residual: the in-process egress broker cannot see sockets a child opens itself — the jail is what closes that (network off $\Rightarrow$ no sockets; a granted child's traffic is not journaled). **Windows has no OS jail** — stdio children there run unconfined, and Shadow says so at startup.

Pinned browser preset (5.5.1): the opt-in Playwright MCP preset (`shadow mcp enable browser`) runs the official `@playwright/mcp` server with an isolated Chrome profile; its output goes to `~/.shadow/playwright-output` (capped), never the workspace. First connect resolves the pinned package from the npm registry via `npx -y` (that resolution IS egress; skipped under `--offline` — see Appendix A #8). A browsing browser needs sockets AND broad filesystem access, so the preset opts this one server out of the jail explicitly (`"network": true`, `"sandbox": false`) — its isolation is the `--isolated` profile + capped output dir. It stays `exec`-gated like every MCP server.

3.9 Offline mode — subtractive by construction

`--offline` requires a local model (or aborts), does **not register** web tools (the model can't call what doesn't exist), skips MCP servers, denies sandbox network egress, and suppresses even the opt-in update check. Mid-session `/model` switches to cloud endpoints are refused. Since v6.7.0 the denial is ALSO enforced below the broker, per runtime: on **Node** the egress broker's enforcing agent is installed as the process-wide undici dispatcher; in the **Bun binary** `globalThis.fetch` is wrapped by the offline wall. Either way a non-local request — including one that bypasses `shadowFetch()` — is refused before dial, and non-local hostnames must resolve to verifiably-local IPs (a spoofed `*.local` name cannot tunnel out). Traffic to the local model server still flows. **Limit**: the fetch-layer wall covers `fetch` only — third-party transports (direct `node:http`, Bun-native sockets in bundled dependencies) are out of scope on both runtimes. The lint guard, the remote-host snapshot test, and the OS sandbox are the layers for those. And the shell-egress denial rides on the OS sandbox — on Windows (no sandbox), `--offline` still drops web tools and MCP but **cannot** stop a shell command from reaching the network.

3.10 Zero telemetry — verifiable

No analytics, crash reporting, or phone-home. The **on-turn** egress: your provider (including the onboarding connection test to the host you are configuring), the web tools when invoked, and the MCP servers you configure. Everything else is **user-initiated only**: (opt-in, off by default, once daily, payload-free) the update check; `shadow update` 's signed-manifest + binary download; the `shadow login` `codex` OAuth scaffold (token exchange unwired); the Context Cooler onboarding install (explicit Y/n); MLX/VLLM first-serve weight downloads; and the plugin paths — an index lookup that exists ONLY if you set `pluginIndexUrl`, and the `shadow plugin add <git-url>` clone that runs only when you run it. Appendix A inventories every one. `shadow doctor --privacy` prints every egress path for the active config, live vs inactive, with **no network calls**. A source-level test (`no-telemetry`) pins the absence of install identifiers AND the exact snapshot of hardcoded remote hosts in `src/` — any new destination fails the build until somebody deliberately reviews it and re-pins the snapshot. **Appendix A** inventories every byte that leaves the machine; the runtime proof is the receipt (`shadow egress / /connections`).

3.11 Untrusted-content envelopes — web + MCP results (P3-05)

Everything Shadow fetches from outside the workspace — `web_fetch` pages, `web_search` snippets, MCP server replies (both transports; success AND error bodies, including server JSON-RPC error messages) — enters the model's context exactly once, wrapped in an envelope: a `[UNTRUSTED_CONTENT - tool · source]` provenance header, a policy line, and `<<<UNTRUSTED_CONTENT_BEGIN>>>` / `<<<UNTRUSTED_CONTENT_END>>>` markers around the payload. The payload that enters the envelope is never mutated (quoting means quoting — a transform an attacker could reason about would be worse than none); for `web_fetch` that payload is the HTML→text reduction of the page, so "untouched" dates from that reduction. Payloads are clamped to the result budget BEFORE enveloping (`fitPayload`), and every downstream cut (the loop's serializer, the post-compact trim) is envelope-safe — a cut that would sever an envelope drops it wholesale instead, so an envelope in the context is always CLOSED. The markers are collision-proof by construction: a payload that contains a marker widens the fence (= padding) until it doesn't, so a forged END cannot terminate the envelope early, and a widened block only closes at the marker matching its own opening padding. The system prompt carries the matching policy (`UNTRUSTED_ENVELOPE_POLICY`) as mechanical glue on every provider and every base-prompt path Shadow assembles — including a user-owned `system_prompt.md` — with one deliberate exception: a `--system / system_prompt_path` override replaces the ENTIRE prompt and gets no glue; containment still holds there because each envelope carries its own inline policy line. This is a context-shaping feature: no new egress, no new processes, pure text formatting. **Guarantee:** for these tool-result payloads, the model can always *tell* outside bytes from workspace text, and no such payload enters the context unwrapped or inside an open envelope (the pre-6.10 MCP `data.content` duplicate did exactly the former). **Limits:** (a) framing is not enforcement — a model that ignores it still acts, and the approval gate + sandbox remain the mechanical backstop (§4.2); (b) truncation sacrifices content to keep containment: an oversized payload is clamped before enveloping (the note is recorded INSIDE the envelope), and a post-compact cut drops a would-be-severed envelope wholesale rather than leaving it open; (c) the MCP tool *schema* is server-controlled and rides every request — it cannot be enveloped (the model must reproduce names and arguments verbatim to call the tool), so it is held by shape rules instead (P2-07): descriptions ARE enveloped and capped (8 KB), tool names are collapsed to the identifier alphabet (a tool cannot be named "ignore previous instructions"), input schemas are capped (32 KB — an oversized tool is skipped, not truncated), and a schema carrying control characters in a key or enum value is rejected fail-closed; (d) workspace files (`read_file`) are not enveloped — the workspace is the user's own data plane, and enveloping every file read would teach the model to distrust the user's own code; (e) `describe_media` returns a model-authored description of a LOCAL image — not outside bytes, so it is not an envelope surface — but the same description currently enters the context twice (`summary` and `data.description`), a context-bloat residual tracked in the ledger; (f) the envelope covers Shadow's *fetch tools* only — **output of shell commands is never enveloped**. A `run_shell` call with network access (allowed by default, §3.2) can `curl / wget` arbitrary web content, and that content returns to the model as raw stdout/stderr — size-clamped, never fenced; `bash_output` drains of a background shell are treated the same. The approval gate (exec + network) and the sandbox are the controls for that path, not the envelope.

3.12 ACP/IDE bridge — the editor as a client

`shadow acp` speaks the Agent Client Protocol (JSON-RPC 2.0, one message per line) over stdin/stdout so an ACP editor — Zed first among them — can drive Shadow as its agent. It is NOT a parallel stack: it reuses the web-console session registry, agent builder, and turn runner, so an editor session inherits the workspace jail (re-resolved from the allowlist on every turn, E1), the

process-wide run lock's FIFO queueing, and the guaranteed terminal `stop` frame on every path. **Wire purity:** stdout is the RPC wire and nothing else — every diagnostic, banner, and vault prompt goes to stderr — so a stray byte cannot corrupt the stream, and the vault's interactive password path can never fire (under an editor stdin is a pipe, never a TTY). A locked vault degrades like `shadow web`: the agent keeps serving, and session builds that need a key fail with a clear error. The wire is scrubbed at the same seam as the web console's stream: every `session/update` payload and every `session/request_permission` params object passes through `redact()` before it leaves the process, so tool inputs, finding bodies, and text deltas cannot carry secret shapes into the editor's persisted thread store (best-effort masking, same posture as §2's other redaction seams). **Approval gate:** where the web console can only deny, the ACP gate bridges each tool call to the editor's `session/request_permission`, so the user decides in the editor (Reject / Allow once / Allow for this session) — but keeps the same fail-closed floor: every ambiguity, dismissal, cancellation, and transport failure resolves to deny. An already-interrupted turn never starts an editor round-trip. **No new egress:** the adapter adds zero outbound surface. Turns reach the provider only through the existing brokered path, and the editor talks to Shadow over stdio inbound-only. **Guarantee:** an editor session cannot widen scope beyond the allowlisted project directory, cannot bypass the approval gate, and cannot leave the wire in an inconsistent state — it inherits every safety property the web console already has. **Limits:** (a) the editor is a trusted local process — a trojaned editor could approve its own prompts, and Shadow has no way to distinguish it (§4.8 posture); (b) `user_question` is auto-answered with each question's first option (v0 editors have no question UI on this channel), so a model's clarifying question does not reach the human — the answers are recorded as a finding; (c) v0 is text-only and has no `session/load`, `set_mode`, or `set_model`, each of which returns a typed "not supported" error rather than silently no-op'ing.

3.13 Sub-agent fan-out — the `agent` tool

The `agent` tool is registered in every session and classified risk `read`: at `auto-read` and above the model launches sub-agents — parallel agent loops with fresh contexts — with **no approval prompt** (only `manual` gates the launch). Fan-out is an amplification axis that sits beside the approval gate, bounded instead by: the session gate and CURRENT autonomy (a sub-agent never escalates — it inherits, so `auto-edit` still asks before its writes/exec); the global `subagentConcurrency` FIFO cap (default 4, max 16; nested fan-out is width-capped per parent); and, since v6.25, budget ceilings — a sub-agent inherits the parent's REMAINING ceilings at admission (zero remainder stops it before its first provider call) and its total spend rolls up to the delegation root on every exit path. Sub-agent contexts are fresh: they see the task prompt, not the parent's transcript — and they DO inherit the workspace jail, gate, and system-prompt policy glue. **Where their instructions come from:** sub-agent definitions load from `~/.shadow/agents` (user), enabled plugins, and `<workspace>/.shadow/agents` (the repo) — **later wins**, so a cloned repo's same-named definition silently overrides yours and a plugin's. A definition's body becomes that sub-agent's system prompt and its `tools` list arms it (§4.14). Skills mirror the same repo-first posture: workspace skill roots are scanned FIRST and first-name-wins, so repo skills outrank plugin skills. **Limit:** an `auto-read` + session therefore runs repo-authored sub-agent prompts without an approval seam — treat `.shadow/` in an untrusted clone the way you treat `shadow.config.json` (§3.7 de-fangs the config; the `agents/skills` dirs are markdown by design and load as content).

4. What Shadow does NOT protect against

Stated with the same weight as the guarantees. If any of these matter to your threat environment, compensate accordingly.

1. **Malicious/steered model output is contained, never prevented.** The gate + sandbox limit what it can *do*, not what it can *try*.
2. **Prompt injection has no mechanical fix.** Since v6.10.0 every untrusted tool-result payload arrives in a structural envelope (§3.11) and the system prompt teaches the model how to read it — but containment is framing, not enforcement: a model that ignores the framing will follow injected instructions right up to the approval gate, which remains the mechanical backstop. Treat `full` autonomy + web tools + sensitive workspace as a risk combination *you* are choosing.
3. **The sandbox fails open** where the OS tool is missing, and does not exist on Windows.
4. **Read-then-exfiltrate:** sandboxed commands can read most of your home directory and reach the network by default. Cut one of the two (`sandboxNetwork: false`, `--offline`) if that's in your threat model.
5. **Redaction is best-effort.** Session logs mask known key shapes and registered secrets; novel secret formats can leak into logs. Redaction is never the reason something is safe to log.
6. **A weak master password bounds the vault.** `script` slows brute force; it does not defeat a dictionary password.

7. **MCP servers are your trust decision.** Since P3-08 stdio children run inside the OS jail on macOS/Linux (network off unless granted with `"network": true`; §3.8); on Windows or hosts with no jail tool they run as your user (`fail-closed` refuses them instead — §3.8). An HTTP MCP endpoint is operator-tier: a configured LAN/internal URL is allowed by design (only the cloud-metadata IP is refused), and the URL is yours, not the model's. Vet what you add.
8. **The provider sees everything you send it** — that is the product working. Local models move this trust to your own hardware and runtime, which you likewise trust.
9. **Your own `~/.shadow` is trusted.** An attacker with home-directory write access owns the agent (and everything else you run).
10. **`--yolo` voids the warranty.** Jail, sandbox, denylist, approvals — all off, by explicit flag.
11. **Supply chain is out of scope of runtime mechanisms.** The installer verifies release signatures (ECDSA-P256, offline key, pinned in the installer); beyond that, the binary, Bun, and npm dependencies are trusted at build/install time.
12. **Side channels, kernel exploits, and a compromised OS** are out of scope entirely.
13. **A plugin you enable is your trust decision.** Plugins are data-only (markdown; executable manifest keys are refused at install; they install DISABLED), but an enabled plugin's markdown becomes prompt content — commands, skills, agent definitions — so a hostile plugin you enable can steer the model like any injected instructions. Review the files before enabling. The optional index is untrusted listings; its signature check is fail-closed when `pluginIndexKey` is set.
14. **Workspace-resident sub-agent definitions and skills are repo-authored prompts with NO install or enable step.** `<workspace> ~/.shadow/agents/*.md` loads automatically, its body becomes that sub-agent's system prompt, its `tools` list arms it, and a same-named repo def OVERRIDES your own (`~/.shadow/agents < plugins < workspace`, later wins; skills are workspace-first, first-name-wins — §3.13). Cloning a repo opts you into its agents. At `auto-read` + the launch needs no approval. Read `~/.shadow/` before working in an untrusted clone.

5. Platform matrix

Capability	macOS	Linux	Windows
OS sandbox for <code>run_shell</code>	✅ seatbelt	✅ bubblewrap	❌ none
OS jail for MCP stdio children	✅ seatbelt + AF_UNIX denies	✅ bwrap + private <code>/tmp</code>	❌ none
<code>~/.shadow</code> invisible to shell	✅	✅ (tmpfs mask)	❌
Denylist patterns	✅ unix	✅ unix	⚠️ unix-oriented — add PowerShell rules via <code>denylistExtra</code>
Keychain for vault key	✅ Keychain	✅ libsecret	⚠️ DPAPI (any process as your user can unprotect)
Workspace jail (file tools)	✅	✅	✅
Offline mode	✅ full	✅ full	⚠️ web/MCP dropped; shell egress not blockable

Windows is not at parity. If your threat model needs OS-level shell containment, run Shadow on macOS or Linux.

6. Operator responsibilities

- **Pick autonomy deliberately.** `auto-edit` is the shipped default; `full` is a trade you make.
- **Use a strong master password** if you use the vault; prefer the keychain over the env var.
- **Vet MCP servers** like you'd vet a shell script you `curl | bash`.
- **On Windows**, add PowerShell denylist rules (`denylistExtra`) and don't assume sandbox behavior.
- **For paranoid workloads:** `--offline` with a local model is the strongest posture Shadow offers — no cloud, no web tools, no MCP, no update check, sandbox egress denied.

7. Verification

- `shadow doctor --privacy` — the live egress report for your config (no network calls).
 - `shadow egress` (or `/connections` in the TUI) — the runtime receipt: every host Shadow reached, for what purpose, allowed or denied. Read from `~/.shadow/egress.log` — works from a fresh process.
 - `shadow doctor` — sandbox/tool availability (so a fail-open never surprises you).
 - The test suite pins the security behaviors above (jail, netguard, denylist, vault crypto, de-fanging, envelopes, no-telemetry) — 2,100+ tests run on every release, and every security-tier release since v6.7.0 has additionally shipped behind a pre-commit adversarial find→verify review (independent finders → per-finding refuters → fix everything confirmed).
 - An internal adversarial audit (14-agent red team; 66 verified findings, all resolved or accepted and documented here) preceded the 2.5.x hardening line. Egress control today is enforced by construction (nothing phones home) and verified by test — not yet by an OS-level egress firewall; that distinction is why this section exists.
 - A 2026-08-07 review (8 mappers → 10 dimension reviewers → 10 adversarial verifiers; 94 findings, 31 CONFIRMED / 3 PARTIAL) re-verified the egress inventory reproduced in Appendix A against on-disk code. Status: zero telemetry **holds**; the gaps were reporting completeness (P1A-17) and the absence of a runtime egress ledger (F00-02) — both closed in v6.7.0 by the egress broker + receipt (P2-01), not violations.
-

Appendix A — Egress inventory (every byte that leaves the machine, verified 2026-08-16)

Reproduced from the Frontier Launch Plan §10. Kept in sync by the host-snapshot guard (`test/no-telemetry.test.ts` , `HOST_SNAPSHOT`) and the broker-only fetch lint rule — since v6.7.0 every item below whose bytes SHADOW'S OWN PROCESS sends routes through `shadowFetch()` (`src/safety/egress.ts`) and lands in the receipt, and since v6.24 the model-authored tool items (3–5) additionally pass the quarantine verdict (§3.4 — flag or deny off-allowlist). Item 6 (`describe_media`) is operator-tier for the quarantine: its destination is the operator-configured `cfg.vision.baseUrl` , not a model-authored URL. **Child-process paths** (not brokered, not always journaled): the plugin git clone (item 16 — bypasses the broker but enforces the offline wall itself and journals its own receipt, purpose `plugin-clone`); the Playwright preset's first-connect `npm -y resolution` (item 8 — an npm-registry fetch from inside the child); the Context Cooler install (item 13 — `git clone + npm install children`); and the MLX/vLLM weight download (item 14 — the `mlx-lm` child talks to huggingface.co). Those are bounded by the offline wall (refused under `--offline`) and, for stdio children, the OS jail — not by the journal.

ALWAYS-on-turn: (1) model provider POST (`stream.ts:287,459` — the streaming request inside `streamWithRetry` and the non-streaming `fetchNonStreamResponse` helper) — user-configured `baseUrl`, conversation body, key in header; NO User-Agent/client-id/metadata. (2) Anthropic count_tokens (`anthropic.ts:122`) — same host, 10s-bounded, anthropic provider only. **ON-TOOL-USE:** (3) `web_fetch` (`webFetch.ts:131`) — arbitrary host, netguard SSRF + DNS-pinned + per-hop redirect re-validation, static non-identifying UA. (4) `web_search` (`webSearch.ts:80`) — `duckduckgo.com/html/`, same guard. (5) remote image fetch (`util/image.ts:31`) — netguard-pinned, 20MiB cap. (6) `describe_media` (`tools/vision.ts:47`) — base64 image to `cfg.vision.baseUrl`, only when configured + not offline. **ON-CONNECT:** (7) MCP HTTP (`client.ts:533,557`) — user-configured url, per-RPC deadline + caller abort. (8) MCP stdio spawn with scrubbedEnv allowlist (no provider creds inherited) — since P3-08 the child is OS-jailed on macOS/Linux with network OFF (plus `AF_UNIX` socket denies / `private/tmp`, §3.8), so its own calls go nowhere by default; a server granted `"network": true` opens sockets the egress broker cannot see (the honest residual — the jail, not the journal, is the layer). The pinned Playwright preset (`network: true, sandbox: false`) runs `npm -y @playwright/mcp@0.0.79` → npm registry at first connect (skipped under `--offline`). **OPT-IN/USER-INITIATED:** (9) update check (`checkUpdate.ts:84`) — default FALSE, once/day, plain GET of public package.json, no params/headers/body, 3s cap, TUI-only. (10) `shadow update binary` (`binary.ts:40`) — signed-manifest-first; asset path reveals OS/arch to `shadow.redpillreader.com`. (11) OAuth scaffold (`oauth.ts:62`) — `auth.openai.com`, only via explicit `shadow login codex`; token exchange unwired. (12) onboarding connection test (`onboard.ts:665`) — user-chosen host via the provider stream path, key registered with the redactor BEFORE the test. (13) Context Cooler onboarding install (`onboard.ts`) — explicit Y/n, git clone + npm install. (14) MLX/vLLM repo-id first-serve weight download — `mlx-lm` subprocess → `huggingface.co`; refused under `--offline`. (15) Plugin index lookup (`registry.ts`) — exists ONLY if `pluginIndexUrl` is set (global-only key; a project file cannot set it), fetched through the broker (purpose `plugin-index`), 1 MB streaming cap, entries coerced + display-sanitized, and when `pluginIndexKey` is set the detached ECDSA P-256 signature is fail-closed. (16) Plugin install clone (`manager.ts:installPluginFromGit`) — only when the user runs `shadow plugin add <git-url>`; scheme allowlist (`https/ssh/file/scp` — `ext::`, `git://`, plain `http://` refused), scrubbed env, `GIT_TERMINAL_PROMPT=0`,

shallow clone into a 0700 temp dir, refused under `--offline` (local `file://` still works); broker-bypass child process, journaled as `plugin-clone` in the receipt. **LOCAL/LAN (not internet egress)**: local-model readiness probes — 127.0.0.1 for Shadow's own auto-serves, the operator-configured host (LAN/mDNS) when the user serves their own endpoint; journaled to the receipt as `local-probe`. Web console (127.0.0.1 bind, CSP `default-src 'none'`), web onboarding (127.0.0.1, 5-min lifetime), and the ACP editor bridge (stdio from the spawning editor, §3.12) are INBOUND-only. No WebSocket/dgram/net.connect/axios/node-fetch anywhere in `src/`. First run after install makes zero calls (offline wizard gate). Session logs redacted per-record, dir 0700/file 0600/.gitignore `**`; provider error bodies pass the redactor before HUD display. One exception to that at-rest posture: `/export` (and `shadow export`) writes the best-effort-redacted transcript as markdown to `<workspace>/exports/` — a normal workspace file, outside the 0700 logs tree, covered by no ignore rule, readable by the model like any workspace file. Redaction is best-effort (§4.5); review or delete exports before committing a workspace.

If you find a gap between this document and the code, that's a bug in one of them — please report it. Honesty here is a feature we ship.